RenAIssance_Showcase

March 20, 2026

0.1 # RenAIssance OCR Pipeline, Complete Project Showcase

0.2 Digitising Early Modern Printed Spanish Documents with Deep Learning

This notebook offers a *complete guide* to the RenAIssance OCR pipeline. The aim is to take scanned PDFs of 16th to 17th-century Spanish printed books and produce high-accuracy digital transcriptions using a multi-stage AI pipeline.

0.2.1 Pipeline Overview

`PDFs → Page Images → YOLO Detection → Crop → Line Segmentation → Alignment → Augmentation → Spl`

0.3 1. Environment Setup & Imports

We import necessary libraries and key functions from each pipeline step.

[56]:
```
import sys, os, json, csv, re, warnings
from pathlib import Path
from collections import defaultdict, Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.ticker import MaxNLocator
from IPython.display import display, HTML, Markdown

# Visualisation settings
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'figure.figsize': (14, 6),
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 12,
    'figure.dpi': 120,
})
warnings.filterwarnings('ignore')
```

```
ROOT = Path('.').resolve()
DATA = ROOT / 'Data'

print(f"Project root: {ROOT}")
print(f"Data folder:  {DATA}")
```

```
Project root: /home/hemanth/Documents/RenAIssance/Test1
Data folder:  /home/hemanth/Documents/RenAIssance/Test1/Data
```

[57]:
```
# importing optional image libraries
try:
    import cv2
    HAS_CV2 = True
except ImportError:
    HAS_CV2 = False
    print("OpenCV not available - some image displays will be skipped.")

try:
    from PIL import Image as PILImage
    HAS_PIL = True
except ImportError:
    HAS_PIL = False

def show_image(path, title="", figsize=(10, 4)):
    """Display an image from a file path."""
    if not HAS_CV2:
        print(f"  [Cannot display: {path}]")
        return
    p = Path(path)
    if not p.exists():
        print(f"  [File not found: {p}]")
        return
    img = cv2.imread(str(p), cv2.IMREAD_UNCHANGED)
    if img is None:
        print(f"  [Unreadable: {p}]")
        return
    if len(img.shape) == 3:
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(1, 1, figsize=figsize)
    ax.imshow(img, cmap='gray' if len(img.shape) == 2 else None)
    ax.set_title(title or p.name, fontweight='bold')
    ax.axis('off')
    plt.tight_layout()
    plt.show()
```

0.4 2. Phase 1 — PDF to Page Images (`step1pdf_to_images.py`)

0.4.1 What this step does

- Converts 6 historical Spanish PDF manuscripts into high-resolution PNG page images, with a minimum of 300 DPI.
- Each PDF is placed in its own named subfolder under `Data/page_images/`.
- Uses **PyMuPDF** for high-quality rasterisation

0.4.2 Key Functions

```
render_pdf_to_pngs(pdf_path, source_name, output_subdir, dpi)  # Renders all pages
list_pdfs(input_dir)                                            # Discovers PDF files
```

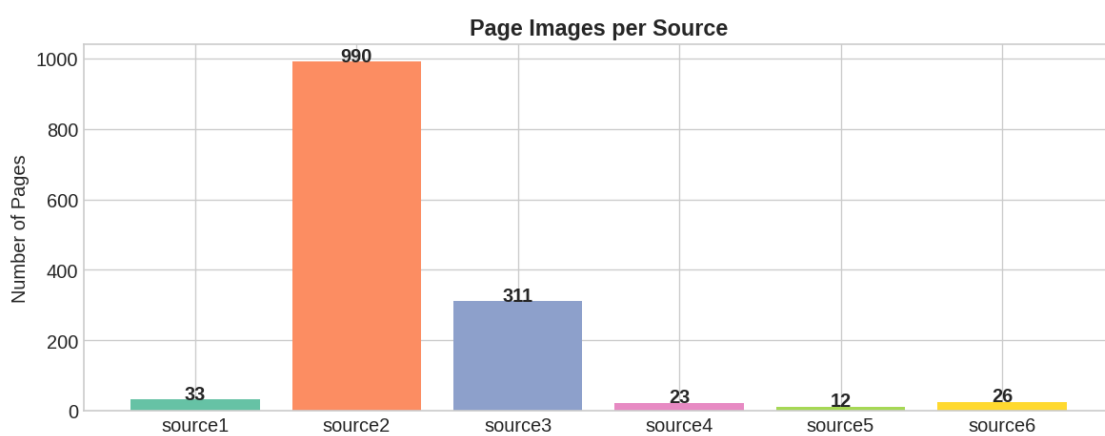
[58]:
```python
# Show the source PDFs
pdf_dir = ROOT / 'Handwritten'
pdfs = sorted(pdf_dir.glob('*.pdf'))
print(f"Source PDFs ({len(pdfs)} documents):\n")
for i, p in enumerate(pdfs, 1):
    size_mb = p.stat().st_size / (1024 * 1024)
    print(f"  {i}. {p.name}  ({size_mb:.1f} MB)")
```

```
Source PDFs (6 documents):

  1. Buendia - Instruccion.pdf  (18.1 MB)
  2. Covarrubias - Tesoro lengua.pdf  (106.6 MB)
  3. Guardiola - Tratado nobleza.pdf  (69.2 MB)
  4. PORCONES.228.38 - 1646.pdf  (45.4 MB)
  5. PORCONES.23.5 - 1628.pdf  (30.5 MB)
  6. PORCONES.748.6 - 1650.pdf  (54.6 MB)
```

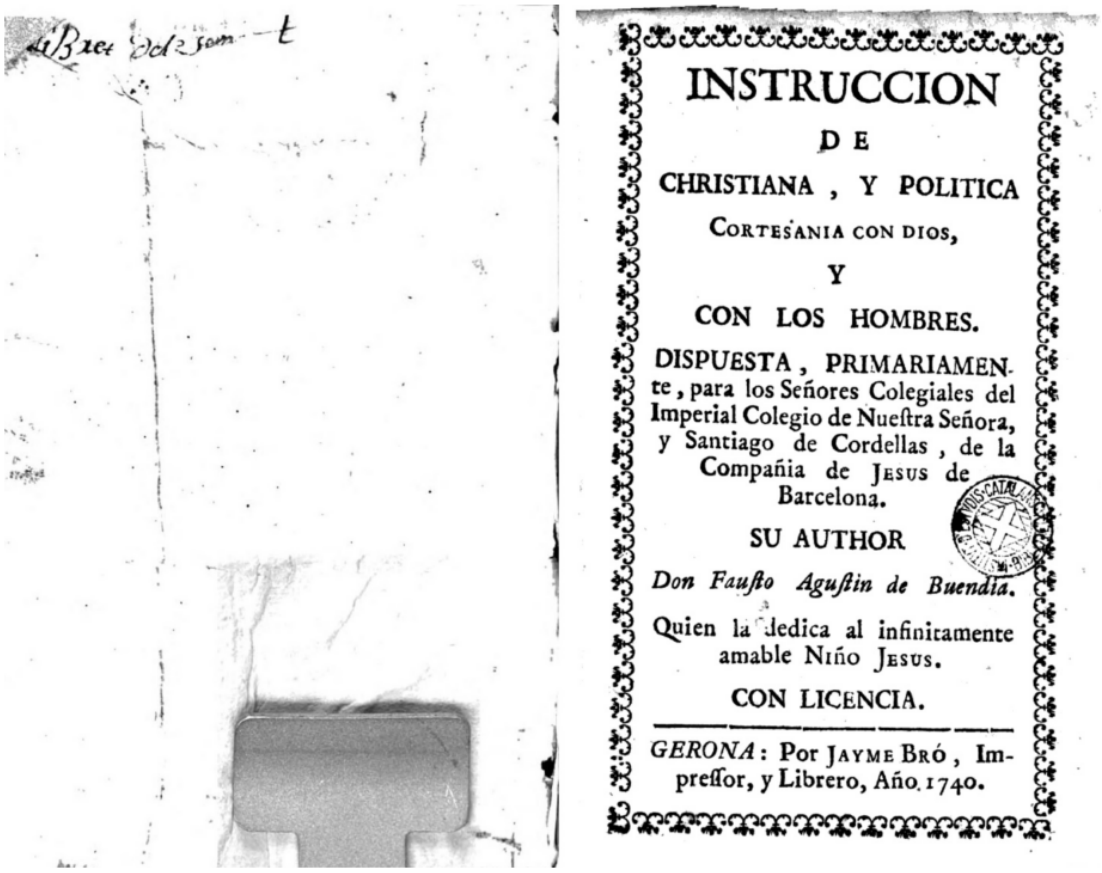
[59]:
```python
# Show generated page images
page_dir = DATA / 'page_images'
if page_dir.exists():
    sources = sorted([d for d in page_dir.iterdir() if d.is_dir()])
    print(f"Page images generated for {len(sources)} sources:\n")
    total = 0
    src_data = []
    for s in sources:
        pages = list(s.glob('*.png'))
        total += len(pages)
        src_data.append({'Source': s.name, 'Pages': len(pages)})
        print(f"  {s.name}: {len(pages)} pages")
    print(f"\n  Total: {total} page images")

    # Bar chart
    df_src = pd.DataFrame(src_data)
    fig, ax = plt.subplots(figsize=(10, 4))
```

```
    bars = ax.bar(df_src['Source'], df_src['Pages'], color=plt.cm.Set2.colors[:
↪len(df_src)])
    ax.set_title('Page Images per Source', fontweight='bold')
    ax.set_ylabel('Number of Pages')
    for bar, val in zip(bars, df_src['Pages']):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,␣
↪str(val),
                ha='center', fontweight='bold')
    plt.tight_layout()
    plt.show()

    # Show a sample page image
    sample = sorted(sources[0].glob('*.png'))[0] if sources else None
    if sample:
        show_image(sample, f"Sample Page Image: {sample.name}", figsize=(8, 10))
else:
    print("Page images directory not found. Run step1pdf_to_images.py first.")
```

```
Page images generated for 6 sources:

  source1: 33 pages
  source2: 990 pages
  source3: 311 pages
  source4: 23 pages
  source5: 12 pages
  source6: 26 pages

  Total: 1395 page images
```



Sample Page Image: source1_page001.png



INSTRUCCION

DE

CHRISTIANA, Y POLITICA

CORTESANIA CON DIOS,

Y

CON LOS HOMBRES.

DISPUESTA, PRIMARIAMENTE, para los Señores Colegiales del Imperial Colegio de Nueſtra Señora, y Santiago de Cordellas, de la Compañia de JESUS de Barcelona.

SU AUTHOR

Don Fauſto Aguſtin de Buendia.

Quien la dedica al infinitamente amable NIÑO JESUS.

CON LICENCIA.

GERONA: Por JAYME BRÓ, Impreſſor, y Librero, Año 1740.

0.5 3. Phase 2 — YOLO Main-Text Detection (`step2_yolo_pipeline.py` + `step3_yolo_crop.py`)

0.5.1 By this step we have achieved the following:

1. **Annotation & Export**: Pages annotated in Label Studio with `main_text` bounding boxes, exported in YOLO format

2. **Stratified Split**: 80/20 train/val split ensuring all 6 sources and both single/multi-column pages appear in both splits

3. **Fine-tune YOLOv8n**: Nano variant, pretrained on COCO, fine-tuned for 1-class detection (`maintext`)
4. **Inference + Crop**: Run on all pages → crop detected text regions

0.5.2 Key Functions

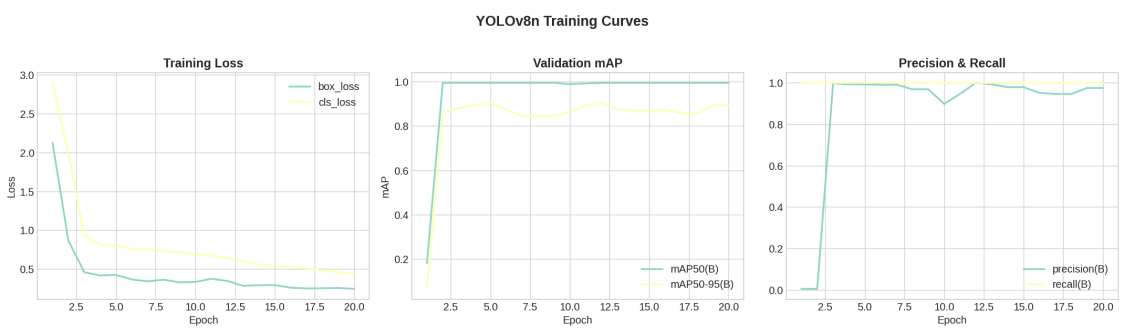
```
stratified_split(labelled_dir, train_ratio, seed)        # Source-aware split
create_yolo_dataset(labelled_dir, output_dir, ...)       # Build YOLO dataset
train_yolo(data_yaml)                                     # Fine-tune YOLOv8n
evaluate_yolo(data_yaml)                                  # Compute mAP, P, R
run_yolo_on_all_pages()                                   # Full-dataset inference
crop_main_text_regions(manifest)                          # Crop detected regions
```

0.5.3 Training Configuration

Parameter	Value	Rationale
Model	YOLOv8n	Lightweight, sufficient for 1-class
Epochs	100 (early stop patience=15)	Prevent overfitting
Batch size	4	4GB VRAM (RTX 3050 Ti)
Image size	480px	Memory-efficient
Augmentation	Minimal (no mosaic/flip)	Documents shouldn't flip

[60]:
```
# Load YOLO training metrics
yolo_results_csv = ROOT / 'runs' / 'yolo_maintext' / 'results.csv'
if yolo_results_csv.exists():
    df_yolo = pd.read_csv(yolo_results_csv)
    df_yolo.columns = [c.strip() for c in df_yolo.columns]

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    # Training loss
    loss_cols = [c for c in df_yolo.columns if 'train/box_loss' in c or 'train/
↪cls_loss' in c]
    if loss_cols:
        for c in loss_cols:
            axes[0].plot(df_yolo['epoch'], df_yolo[c], label=c.split('/')[-1],␣
↪linewidth=2)
        axes[0].set_title('Training Loss', fontweight='bold')
        axes[0].set_xlabel('Epoch')
        axes[0].set_ylabel('Loss')
        axes[0].legend()

    # mAP
    map_cols = [c for c in df_yolo.columns if 'metrics/mAP' in c]
    if map_cols:
        for c in map_cols:
            axes[1].plot(df_yolo['epoch'], df_yolo[c], label=c.split('/')[-1],␣
↪linewidth=2)
        axes[1].set_title('Validation mAP', fontweight='bold')
        axes[1].set_xlabel('Epoch')
        axes[1].set_ylabel('mAP')
```

```
        axes[1].legend()

    # Precision & Recall
    pr_cols = [c for c in df_yolo.columns if 'precision' in c.lower() or␣
↪'recall' in c.lower()]
    if pr_cols:
        for c in pr_cols:
            axes[2].plot(df_yolo['epoch'], df_yolo[c], label=c.split('/')[-1],␣
↪linewidth=2)
        axes[2].set_title('Precision & Recall', fontweight='bold')
        axes[2].set_xlabel('Epoch')
        axes[2].legend()

    plt.suptitle('YOLOv8n Training Curves', fontsize=16, fontweight='bold', y=1.
↪02)
    plt.tight_layout()
    plt.show()
else:
    print("  YOLO results.csv not found. Training curves unavailable.")
    print("  Expected at:", yolo_results_csv)
```



[61]:
```
# Show crop statistics
crop_manifest_path = DATA / 'crop_manifest.json'
if crop_manifest_path.exists():
    with open(crop_manifest_path) as f:
        crop_manifest = json.load(f)

    print("YOLO Crop Statistics\n")
    print(f"  Total crops created:     {crop_manifest.get('total_crops', 'N/
↪A')}")
    print(f"  Pages with no detection: {crop_manifest.
↪get('skipped_pages_no_detection', 'N/A')}")
    print(f"  Padding (px):            {crop_manifest.get('pad_px', 'N/A')}")
    print(f"\n  Crops per source:")
```

```
    cps = crop_manifest.get('crops_per_source', {})
    for src, cnt in sorted(cps.items()):
        print(f"    {src}: {cnt}")

    # Show a sample crop
    crops_dir = DATA / 'cropped_main_text'
    if crops_dir.exists():
        sample_sources = sorted([d for d in crops_dir.iterdir() if d.is_dir()])
        if sample_sources:
            samples = sorted(sample_sources[0].glob('*.png'))[:1]
            for s in samples:
                show_image(s, f"Sample Cropped Text Region: {s.name}",␣
 ↪figsize=(12, 6))
else:
    print("Crop manifest not found.")
```

```
YOLO Crop Statistics

  Total crops created:     2403
  Pages with no detection: 35
  Padding (px):            5

  Crops per source:
    source1: 61
    source2: 1950
    source3: 298
    source4: 32
    source5: 23
    source6: 39
```

Sample Cropped Text Region: source1_page001_full.png

INSTRUCCION

DE

CHRISTIANA, Y POLITICA

CORTESANIA CON DIOS,

Y

CON LOS HOMBRES.

DISPUESTA, PRIMARIAMENte, para los Señores Colegiales del Imperial Colegio de Nueſtra Señora, y Santiago de Cordellas, de la Compañia de JESUS de Barcelona.

SU AUTHOR

Don Fauſto Aguſtin de Buendia.

Quien la dedica al infinitamente amable NIÑO JESUS.

CON LICENCIA.

GERONA: Por JAYME BRÓ, Impreſſor, y Librero, Año 1740.

0.6 4. Phase 3 -> Line Segmentation (`step4_line_segmentation.py`)

0.6.1 What this step does

Per cropped text region: 1. **Deskew** — projection-profile variance maximisation (two-pass: coarse 0.5° + fine 0.1°) 2. **Denoise / Remove Bleed-Through** — median filter + morphological background normalisation 3. **Binarize** — Sauvola adaptive thresholding (handles uneven illumination) 4. **Seam-Carving Line Segmentation** — constrained DP between projection valleys, energy = ink density + distance transform 5. **Crop** individual line strip images with white-padded seam boundaries

0.6.2 Why Seam Carving?

Historical scans have **curved baselines** and **bleed-through** where simple horizontal cuts fail. Seam carving traces the natural boundary between lines using dynamic programming.

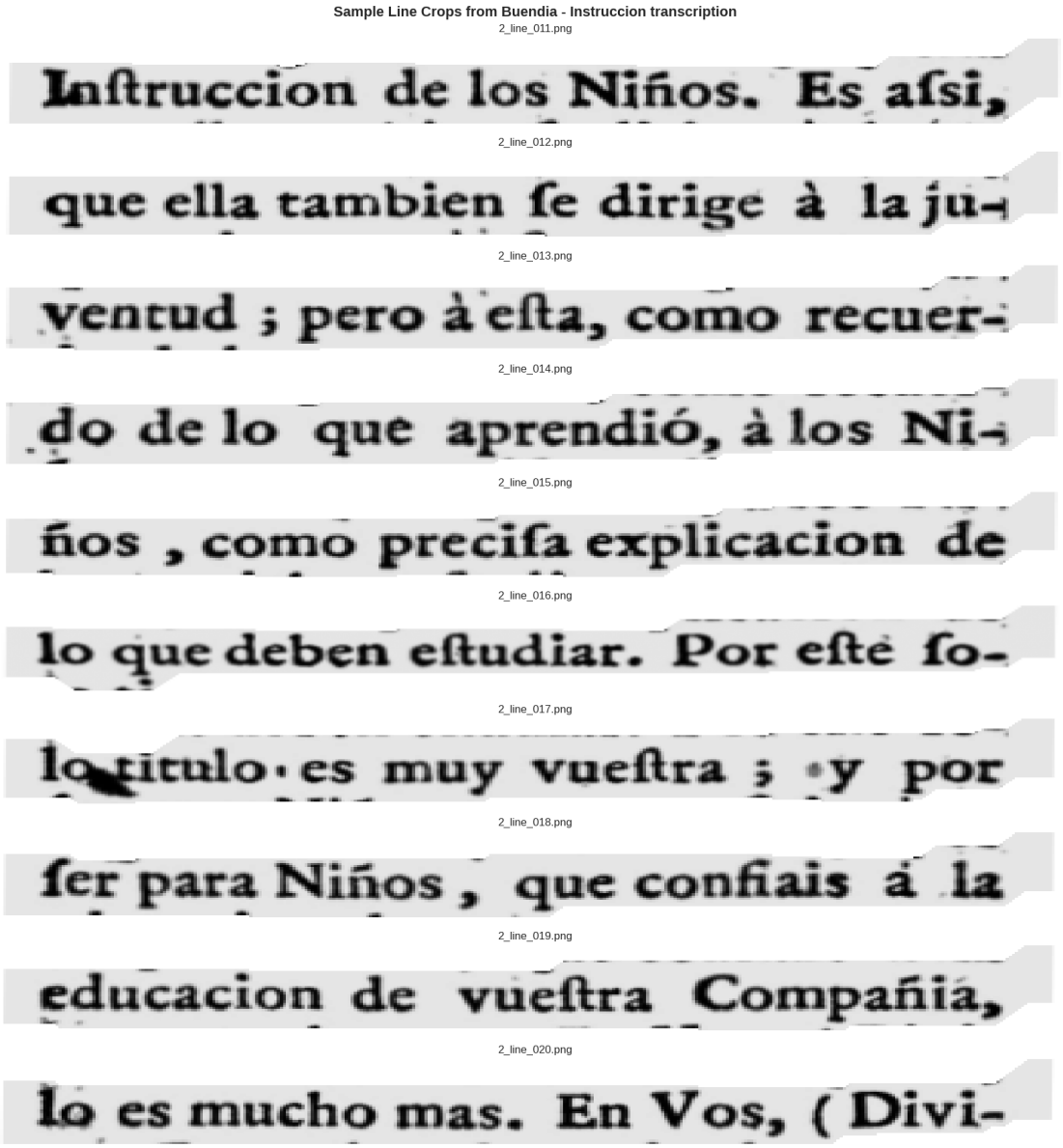
0.6.3 Key Functions

```
estimate_skew_angle(gray)              # Two-pass angle estimation
deskew(gray)                           # Deskew using skimage rotation
denoise_and_remove_bleed(gray)         # Median + morphological normalisation
binarize(gray, window_size, k)         # Sauvola thresholding
segment_lines(binary)                  # Seam-carving line detection
crop_line(image, region, pad_y)        # Seam-bounded line extraction
compute_ink_ratio(line_img)            # Dark-pixel ratio for filtering
parse_transcript(docx_path)            # .docx → ordered text lines
preprocess_and_segment(image_path, output_dir, ...)  # Full per-image pipeline
```

[62]:
```
# Show line crops and segmentation stats
line_crops_dir = DATA / 'line_crops'
if line_crops_dir.exists():
    source_dirs = sorted([d for d in line_crops_dir.iterdir() if d.is_dir()])
    print(f"Line Segmentation Results ({len(source_dirs)} sources)\n")

    lc_data = []
    for sd in source_dirs:
        lines = list(sd.glob('*.png'))
        lc_data.append({'Source': sd.name[:30], 'Lines': len(lines)})
        print(f"  {sd.name}: {len(lines)} line crops")

    total_lines = sum(d['Lines'] for d in lc_data)
    print(f"\n  Total line crops: {total_lines}")

    # Show sample line crops from first source
    if source_dirs:
        sample_lines = sorted(source_dirs[0].glob('*.png'))[10:20]
        if sample_lines:
            fig, axes = plt.subplots(len(sample_lines), 1, figsize=(14,␣
↪len(sample_lines)*1.5))
            if len(sample_lines) == 1:
                axes = [axes]
            for ax, lp in zip(axes, sample_lines):
                if HAS_CV2:
                    img = cv2.imread(str(lp), cv2.IMREAD_GRAYSCALE)
                    if img is not None:
                        ax.imshow(img, cmap='gray', aspect='auto')
                ax.set_title(lp.name, fontsize=10)
                ax.axis('off')
            plt.suptitle(f'Sample Line Crops from {source_dirs[0].name}',
```

```
                        fontweight='bold', fontsize=13)
            plt.tight_layout()
            plt.show()
else:
    print("Line crops directory not found.")
```

```
Line Segmentation Results (6 sources)

  Buendia - Instruccion transcription: 107 line crops
  Covarrubias - Tesoro lengua transcription: 129 line crops
  Guardiola - Tratado nobleza transcription: 79 line crops
  PORCONES.228.38 - 1646: 163 line crops
  PORCONES.23.5 - 1628: 190 line crops
  PORCONES.748.6 - 1650: 130 line crops

  Total line crops: 798
```



0.7 5. Phase 4 — Ground Truth Alignment (`step5_rename_and_realign_txt.py`)

0.7.1 What this step does

- Parse transcripts (`.txt` / `.docx`) into ordered text lines
- Align line crop images with transcript lines **sequentially** (reading order)
- Handle multi-column pages (left column before right)
- Output a unified CSV: `source, page, line, image_path, ground_truth_text`

0.7.2 Alignment Strategy

```
Line crop 001  →  "Al"
```

```
Line crop 002  →  "INFINITAMENTE AMABLE"
Line crop 003  →  "NIÑO JESUS."
...
```

[63]:
```python
import csv
import random
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import matplotlib.patches as mpatches
from pathlib import Path

ALIGNMENT_CSV = DATA / 'line_alignment.csv'
N_SAMPLES     = 6
RANDOM_SEED   = 42

# Load aligned rows
rows = []
with open(ALIGNMENT_CSV, encoding='utf-8-sig') as f:
    for row in csv.DictReader(f):
        img_path = row.get('image_path', '').strip()
        gt_text  = row.get('ground_truth_text', '').strip()
        status   = row.get('status', 'aligned').strip().lower()
        if img_path and gt_text and status == 'aligned' and Path(img_path).
 ↪exists():
            rows.append(row)

print(f"Loaded {len(rows)} aligned rows")

# Sample
random.seed(RANDOM_SEED)
samples = random.sample(rows, min(N_SAMPLES, len(rows)))
fig, axes = plt.subplots(len(samples), 1, figsize=(14, 3.2 * len(samples)))
fig.patch.set_facecolor('#F7F5F0')

if len(samples) == 1:
    axes = [axes]

for i, (ax, row) in enumerate(zip(axes, samples)):
    ax.set_facecolor('#1a1a1a')

    #   Image
    img = mpimg.imread(row['image_path'])
    ax.imshow(img, cmap='gray' if img.ndim == 2 else None,
              aspect='auto', extent=[0, 1, 0, 1])
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.set_xticks([])
```

```
    ax.set_yticks([])

    #   Border around image
    for spine in ax.spines.values():
        spine.set_edgecolor('#CCBBAA')
        spine.set_linewidth(1.2)
    ax.text(
        0.0, 1.07, "Ground Truth",
        transform=ax.transAxes,
        fontsize=9, fontweight='bold',
        color='white',
        bbox=dict(boxstyle='round,pad=0.3', facecolor='#3a5a3a',␣
↪edgecolor='none'),
        va='bottom', ha='left'
    )


    ax.text(
        0.145, 1.075, f'"{gt_text}"',
        transform=ax.transAxes,
        fontsize=10.5, fontstyle='italic',
        color='#1a1a1a',
        va='bottom', ha='left',
        clip_on=False
    )

    source = row.get('source', 'Unknown')
    page   = row.get('page', '?')
    line   = row.get('line', '?')
    ax.text(
        1.0, -0.04, f"Source: {source}   Page: {page}   Line: {line}",
        transform=ax.transAxes,
        fontsize=8.5, color='#666655',
        va='top', ha='right'
    )

    ax.text(
        0.01, 0.96, f"#{i+1}",
        transform=ax.transAxes,
        fontsize=8, fontweight='bold', color='white',
        bbox=dict(boxstyle='round,pad=0.25', facecolor='#444',␣
↪edgecolor='none', alpha=0.75),
        va='top', ha='left'
    )

fig.suptitle(
    "Line Alignment Visualization - Image     Ground Truth",
```

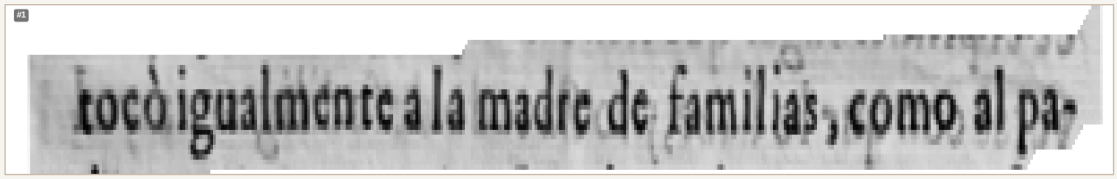
```
    fontsize=14, fontweight='bold', color='#1a1a1a',
    y=1.01
)

plt.tight_layout(h_pad=3.5)
plt.show()
```

```
Loaded 773 aligned rows
```

Line Alignment Visualization — Image ↔ Ground Truth

Ground Truth *"mía, ser así tan indiscreta."*

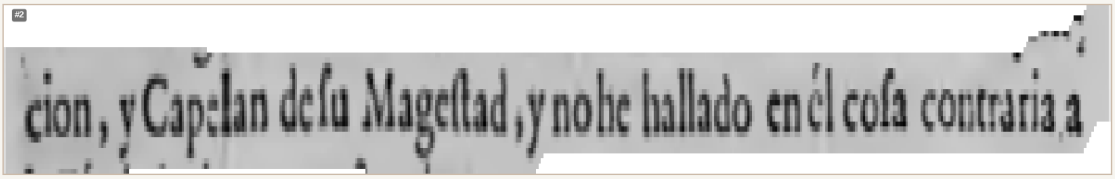
#1



Source: PORCONES.23.5 - 1628 Page: 4left Line: 178

Ground Truth *"mía, ser así tan indiscreta."*

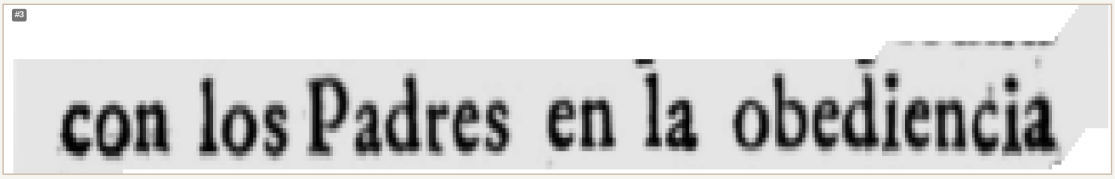
#2



Source: Covarrubias - Tesoro lengua transcription Page: 7 Line: 9

Ground Truth *"mía, ser así tan indiscreta."*

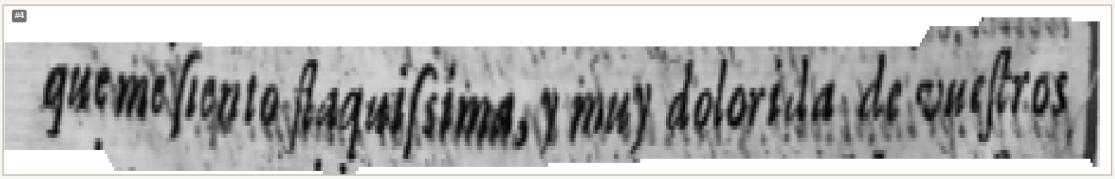
#3



Source: Buendia - Instruccion transcription Page: 3 Line: 26

Ground Truth *"mía, ser así tan indiscreta."*

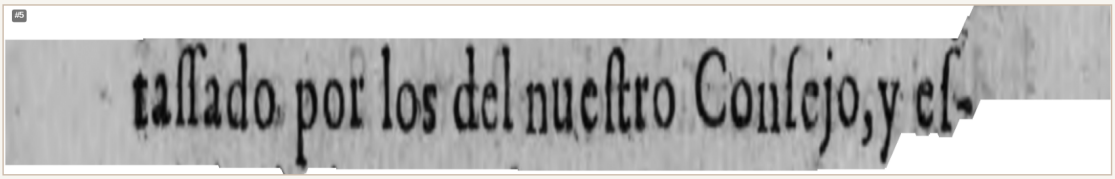
#4



Source: PORCONES.748.6 – 1650 Page: 3 Line: 93

Ground Truth *"mía, ser así tan indiscreta."*

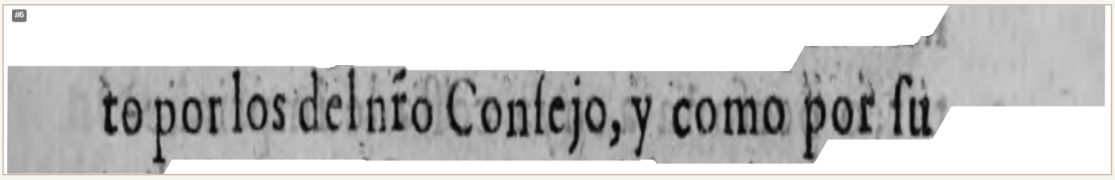
#5



Source: Guardiola - Tratado nobleza transcription Page: 13 Line: 47

Ground Truth *"mía, ser así tan indiscreta."*

#6



Source: Guardiola - Tratado nobleza transcription Page: 12 Line: 16

0.8 6. Phase 5 & 6 — Augmentation & Source-Level Split (`step6_augmentation_split.py`)

0.8.1 Split Strategy

- **Source-level split** (not line-level) — tests real generalisation to unseen books
- 4 sources → Training, 2 sources → Validation

0.8.2 Augmentations Applied (training only)

Augmentation	Why
Rotation ±2°	Simulates tilted scanning
Gaussian blur	Simulates focus issues
Brightness ±12	Simulates ink fading
Gaussian noise [2,8]	Simulates paper texture
Elastic distortion	Simulates paper warping

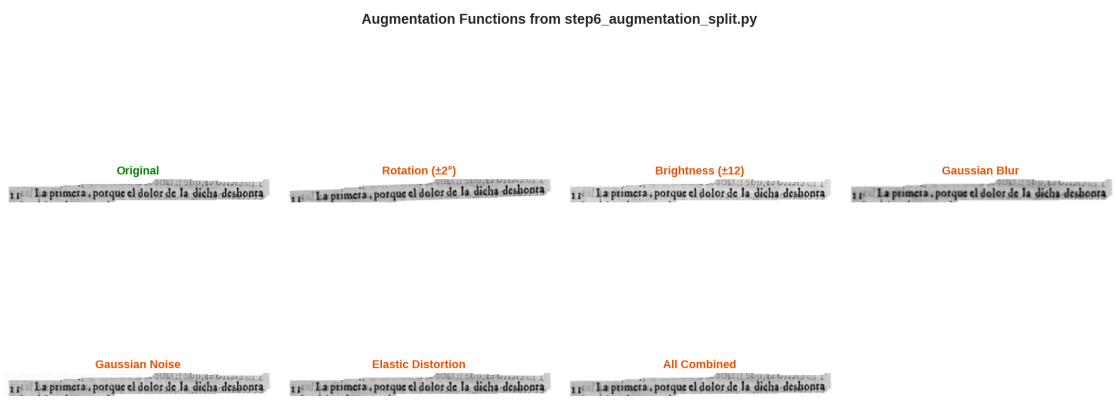
0.8.3 Key Functions

```
apply_rotation(img, rng)        apply_blur(img, rng)
apply_brightness(img, rng)      apply_noise(img, rng)
apply_elastic(img, rng)         augment_image(img_gray, rng)
pick_val_sources(...)           read_alignment_rows(csv)
```

[64]:
```
#   Import augmentation functions from step6_augmentation_split.py
import sys, os, glob, random
sys.path.insert(0, str(ROOT))  # ensure project root is on path

from step6_augmentation_split import (
    apply_rotation,
    apply_brightness,
    apply_blur,
    apply_noise,
    apply_elastic,
    augment_image,
)

#   Pick a random line crop image
line_crops_dir = DATA / 'line_crops'
all_crops = sorted(glob.glob(str(line_crops_dir / '**' / '*.png'),␣
 ↪recursive=True))
assert len(all_crops) > 0, 'No line crop images found!'

random.seed(42)  # reproducible
chosen_crop = random.choice(all_crops)
print(f'Selected line crop: {chosen_crop}')

# Read as grayscale (augmentation functions expect uint8 grayscale)
```

```python
original = cv2.imread(chosen_crop, cv2.IMREAD_GRAYSCALE)
assert original is not None, f'Could not read {chosen_crop}'

#   Apply each augmentation individually
rng = np.random.default_rng(123)

augmentations = {
    'Original':           original,
    'Rotation (±2°)':     apply_rotation(original, np.random.default_rng(10)),
    'Brightness (±12)':   apply_brightness(original, np.random.
 ↪default_rng(100)),
    'Gaussian Blur':      apply_blur(original, np.random.default_rng(30)),
    'Gaussian Noise':     apply_noise(original, np.random.default_rng(40)),
    'Elastic Distortion': apply_elastic(original, np.random.default_rng(50)),
    'All Combined':       augment_image(original, np.random.default_rng(99)),
}

#   Visualise the results in a 2-row grid
n = len(augmentations)
n_cols = 4
n_rows = (n + n_cols - 1) // n_cols  # ceil division

fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes = axes.flatten()

for idx, (title, img) in enumerate(augmentations.items()):
    ax = axes[idx]
    ax.imshow(img, cmap='gray')
    ax.set_title(title, fontweight='bold', fontsize=13,
                 color='green' if title == 'Original' else '#E65100')
    ax.axis('off')

# Hide any unused subplot slots
for idx in range(n, len(axes)):
    axes[idx].axis('off')

fig.suptitle('Augmentation Functions from step6_augmentation_split.py',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

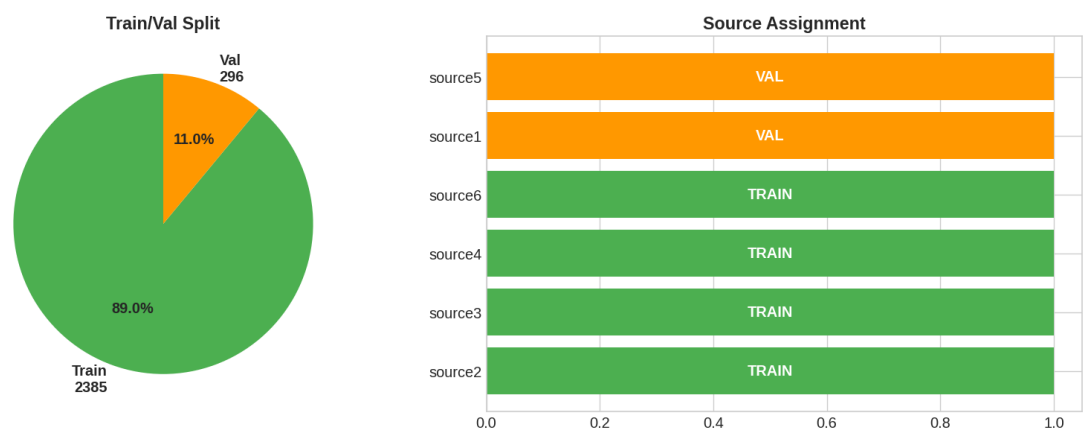
```
Selected line crop:
/home/hemanth/Documents/RenAIssance/Test1/Data/line_crops/PORCONES.23.5 -
1628/4left_line_022.png
```



[65]:
```
# Display split statistics
split_stats_path = DATA / 'ocr_dataset' / 'split_stats.json'
if split_stats_path.exists():
    with open(split_stats_path) as f:
        sstats = json.load(f)

    print("  Train/Val Split Statistics\n")
    print(f"  Train sources: {sstats['train_source_ids']}")
    print(f"  Val sources:   {sstats['val_source_ids']}")
    print(f"  Augmentations per image: {sstats['aug_per_image']}")
    print(f"\n  Train rows (augmented): {sstats['train_rows']}")
    print(f"  Val rows (pristine):    {sstats['val_rows']}")
    print(f"  Total rows:             {sstats['full_rows']}")

    # Pie chart
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    axes[0].pie([sstats['train_rows'], sstats['val_rows']],
                labels=[f"Train\n{sstats['train_rows']}",␣
 ↪f"Val\n{sstats['val_rows']}"],
                colors=['#4CAF50', '#FF9800'], autopct='%1.1f%%',
                startangle=90, textprops={'fontsize': 12, 'fontweight': 'bold'})
    axes[0].set_title('Train/Val Split', fontweight='bold')

    # Source distribution
    src_labels = sstats['train_source_ids'] + sstats['val_source_ids']
    src_colors = ['#4CAF50']*len(sstats['train_source_ids']) +␣
 ↪['#FF9800']*len(sstats['val_source_ids'])
    axes[1].barh(src_labels, [1]*len(src_labels), color=src_colors)
```

```
    axes[1].set_title('Source Assignment', fontweight='bold')
    axes[1].set_xlabel('')
    for i, sid in enumerate(src_labels):
        label = 'TRAIN' if sid in sstats['train_source_ids'] else 'VAL'
        axes[1].text(0.5, i, label, ha='center', va='center',␣
↪fontweight='bold', fontsize=12, color='white')

    plt.tight_layout()
    plt.show()
else:
    print("Split stats not found.")
```

```
Train/Val Split Statistics

 Train sources: ['source2', 'source3', 'source4', 'source6']
 Val sources:   ['source1', 'source5']
 Augmentations per image: 5

 Train rows (augmented): 2385
 Val rows (pristine):    296
 Total rows:             2681
```



0.9 7. Phase 7 — Tesseract Baseline (`step7_tesseract_baseline.py`)

0.9.1 What this step does

- Run **out-of-the-box Tesseract** (Spanish, PSM 6) on validation line images
- Compute CER, WER, chrF — this is the **floor** we must beat

0.9.2 Why Tesseract?

Tesseract provides a strong, reproducible baseline. It has no knowledge of early modern Spanish or historical fonts, so any improvement by our fine-tuned models demonstrates genuine learning.

0.9.3 Key Functions

```
run_tesseract(image_path, lang, psm, oem)   # Single-image OCR
levenshtein_distance(seq_a, seq_b)           # Edit distance
cer_stats(ref, hyp)                          # Character Error Rate
wer_stats(ref, hyp)                          # Word Error Rate
chrf_corpus(refs, hyps)                      # Character n-gram F-score
aggregate_metrics(rows)                      # Corpus-level metrics
```

0.10 8. Phase 8 — TrOCR Fine-tuning (`step8_trocr_finetune.py`)

0.10.1 What this step does

- Fine-tune **microsoft/trocr-base-printed** (Vision Encoder-Decoder) on our aligned pairs
- Frozen encoder (only decoder trained) - Because of Computation Constraints
- fp16 mixed precision + gradient checkpointing + 8-bit AdamW
- Early stopping on validation CER

0.10.2 Architecture

```
[Line Image] → ViT Encoder (frozen) → Cross-Attention → GPT-2 Decoder → [Text]
```

0.11 9. Phase 9 — Kraken CRNN (`step9.0-9.2`)

0.11.1 Three sub-steps

1. **Prepare Data** (`step9.0`): Convert manifests to Kraken ground-truth format (image + `.gt.txt` pairs)
2. **Compile** (`step9.1`): Compile into binary Arrow datasets using `ketos compile`
3. **Train** (`step9.2`): Train CRNN with cosine schedule, augmentation, auto batch-size fallback

0.11.2 Why CRNN alongside TrOCR?

- **Different architecture** -> different error patterns -> complementary strengths
- CRNNs are faster at inference and can handle variable-width inputs natively
- Kraken is purpose-built for historical document OCR

0.12 10. Phase 10 — LLM Post-Correction (`step10_Post_LLM_correction.py`)

0.12.1 What this step does

- Send raw CRNN output to **Gemini 2.5 Flash** in JSON chunks
- Strict prompt: fix **only** character-level OCR errors, NO modernisation, NO hallucination
- Preserve archaic spellings, hyphens, punctuation exactly

0.12.2 Prompt Design Highlights

- No accent modernisation (except ñ)
- No word completion across lines
- Fix f/s, I/l, rn/m confusions
- ç → z, expand q → que
- Temperature = 0.0 for deterministic output

0.13 11. Final Results & Comprehensive Analysis

[66]:
```
# Load the final LLM-corrected results
llm_csv = ROOT / 'llm_corrected_results_final.csv'
kraken_csv = ROOT / 'kraken_eval_results.csv'

if llm_csv.exists():
    df_final = pd.read_csv(llm_csv)
    print(f"Loaded LLM-corrected results: {len(df_final)} samples")
    print(f"  Columns: {list(df_final.columns)}")
    display(df_final.head(3))
else:
    print("LLM results file not found")
    df_final = None

if kraken_csv.exists():
    df_kraken = pd.read_csv(kraken_csv)
    print(f"\n Loaded Kraken eval results: {len(df_kraken)} samples")
else:
    df_kraken = None
```

```
Loaded LLM-corrected results: 296 samples
  Columns: ['image_path', 'ground_truth_text', 'crnn_prediction',
'cer_per_line', 'llm_correction', 'cer_before', 'cer_after', 'wer_before',
'wer_after', 'chrf_before', 'chrf_after']

                                          image_path     ground_truth_text  \
0  /home/hemanth/Documents/RenAIssance/Test1/Data…                    Al
1  /home/hemanth/Documents/RenAIssance/Test1/Data…  INFINITAMENTE AMABLE
2  /home/hemanth/Documents/RenAIssance/Test1/Data…           NIÑO JESUS.

     crnn_prediction  cer_per_line     llm_correction  cer_before  cer_after  \
0                 AR      0.500000                 Al    0.666667   0.666667
1  NEINTTAMRNTE AMAE      0.300000  ATENTAMENTE AMADO    0.300000   0.400000
2              NNO B      0.727273              NNO B    0.727273   0.727273

   wer_before  wer_after  chrf_before  chrf_after
0         1.0        1.0     0.057471    0.057471
1         1.0        1.0     0.316695    0.474702
2         1.0        1.0     0.035461    0.035461
```

```
Loaded Kraken eval results: 296 samples
```

[67]:
```python
# Trocr Finetune
import pandas as pd
import numpy as np
import sys
import subprocess

# Install metrics packages if missing
try:
    import jiwer
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "jiwer",␣
 ↪"-q"])
    import jiwer

try:
    from sacrebleu.metrics import CHRF
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "sacrebleu",␣
 ↪"-q"])
    from sacrebleu.metrics import CHRF

# Load CRNN evaluation results
results_path = "/home/hemanth/Documents/RenAIssance/Test1/runs/trocr_finetune/
 ↪predictions.csv"
eval_df = pd.read_csv(results_path)

# Keep only needed columns and clean missing values
eval_df = eval_df[["ground_truth_text", "prediction"]].copy()
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].fillna("").
 ↪astype(str)
eval_df["prediction"] = eval_df["prediction"].fillna("").astype(str)

# Remove BOM if present in text
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].str.
 ↪replace("\ufeff", "", regex=False)
eval_df["prediction"] = eval_df["prediction"].str.replace("\ufeff", "",␣
 ↪regex=False)

refs = eval_df["ground_truth_text"].tolist()
hyps = eval_df["prediction"].tolist()

# Per-line metrics (optional but useful for analysis)
eval_df["cer_recalc"] = [jiwer.cer(r, h) for r, h in zip(refs, hyps)]
eval_df["wer_recalc"] = [jiwer.wer(r, h) for r, h in zip(refs, hyps)]
```

```
# Corpus-level metrics
corpus_cer = jiwer.cer("\n".join(refs), "\n".join(hyps))
corpus_wer = jiwer.wer("\n".join(refs), "\n".join(hyps))
chrf_metric = CHRF()
corpus_chrf = chrf_metric.corpus_score(hyps, [refs]).score

print(f"Rows evaluated: {len(eval_df)}")
print(f"CER (corpus):  {corpus_cer:.6f}")
print(f"WER (corpus):  {corpus_wer:.6f}")
print(f"chrF (corpus): {corpus_chrf:.4f}")

display(eval_df[["ground_truth_text", "prediction", "cer_recalc",␣
↪"wer_recalc"]].head())
```

```
Rows evaluated: 296
CER (corpus):  0.174957
WER (corpus):  0.465684
chrF (corpus): 65.0673
```

	ground_truth_text	prediction	cer_recalc	wer_recalc
0	Al	ALT	1.000000	1.0
1	INFINITAMENTE AMABLE	INFINITAMENT DE	0.350000	1.0
2	NIÑO JESUS.	NINO JESUS.	0.090909	0.5
3	A Vos, Dulcissimo Niño	vos, Dulosimo Nió	0.363636	1.0
4	JESUS, que no solo os	fosUS, que no solo os que	0.333333	0.4

[68]:
```
# CRNN (Kraken)
import pandas as pd
import numpy as np
import sys
import subprocess

# Install metrics packages if missing
try:
    import jiwer
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "jiwer",␣
↪"-q"])
    import jiwer

try:
    from sacrebleu.metrics import CHRF
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "sacrebleu",␣
↪"-q"])
    from sacrebleu.metrics import CHRF
```

```python
# Load CRNN evaluation results
results_path = "/home/hemanth/Documents/RenAIssance/Test1/kraken_eval_results.
↪csv"
eval_df = pd.read_csv(results_path)

# Keep only needed columns and clean missing values
eval_df = eval_df[["ground_truth_text", "crnn_prediction"]].copy()
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].fillna("").
↪astype(str)
eval_df["crnn_prediction"] = eval_df["crnn_prediction"].fillna("").astype(str)

# Remove BOM if present in text
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].str.
↪replace("\ufeff", "", regex=False)
eval_df["crnn_prediction"] = eval_df["crnn_prediction"].str.replace("\ufeff",␣
↪"", regex=False)

refs = eval_df["ground_truth_text"].tolist()
hyps = eval_df["crnn_prediction"].tolist()

# Per-line metrics (optional but useful for analysis)
eval_df["cer_recalc"] = [jiwer.cer(r, h) for r, h in zip(refs, hyps)]
eval_df["wer_recalc"] = [jiwer.wer(r, h) for r, h in zip(refs, hyps)]

# Corpus-level metrics
corpus_cer = jiwer.cer("\n".join(refs), "\n".join(hyps))
corpus_wer = jiwer.wer("\n".join(refs), "\n".join(hyps))
chrf_metric = CHRF()
corpus_chrf = chrf_metric.corpus_score(hyps, [refs]).score

print(f"Rows evaluated: {len(eval_df)}")
print(f"CER (corpus):  {corpus_cer:.6f}")
print(f"WER (corpus):  {corpus_wer:.6f}")
print(f"chrF (corpus): {corpus_chrf:.4f}")

display(eval_df[["ground_truth_text", "crnn_prediction", "cer_recalc",␣
↪"wer_recalc"]].tail())
```

```
Rows evaluated: 296
CER (corpus):  0.092784
WER (corpus):  0.333487
chrF (corpus): 78.3699
```

```
                                    ground_truth_text  \
291  sar la injuria hecha a su casa, ex Alex. ad Ba…
292  item ait, ff. de iniurijs: vbi vero, la ofensa…
293  marido y muger, lo es tambien el derecho de ac…
294  puede la muger suplir la omission del marido, …
```

```
295  bro in §. patitur, nu. 4. inst. de iniurijs, F…

                                        crnn_prediction  cer_recalc  wer_recalc
291  sar la injuria hecha a su casa, ex dles ad Bar…     0.163934    0.266667
292  titem ait, sie de iniurijg,: vbi vero, la oste…     0.140351    0.333333
293  marido y muger, lo es tambien el derecho de ae…     0.018868    0.090909
294  puede la muger suplir la omission del marido, …     0.038462    0.200000
295  bro in S. patitur, nu 4. inst, de iniuria. Far…     0.131148    0.500000
```

[69]:

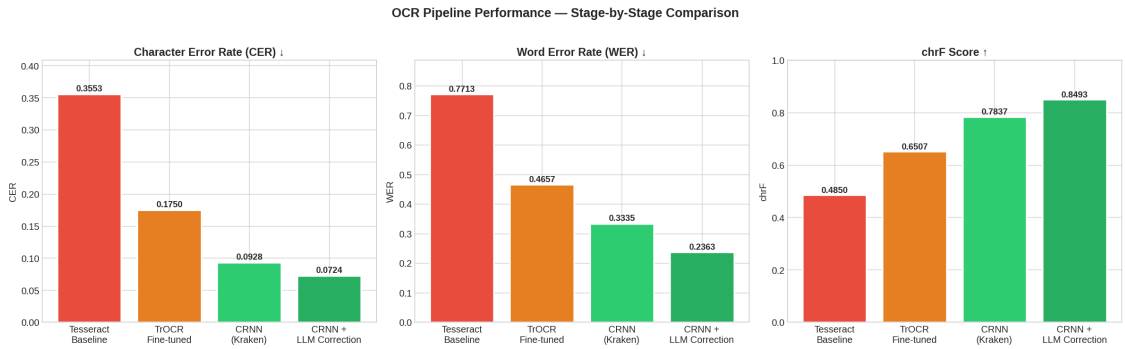
```python
import pandas as pd
import sys
import subprocess

# Install metrics packages if missing
try:
    import jiwer
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "jiwer",␣
 ↪"-q"])
    import jiwer

try:
    from sacrebleu.metrics import CHRF
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "sacrebleu",␣
 ↪"-q"])
    from sacrebleu.metrics import CHRF

# Load post-LLM correction results
results_path = "/home/hemanth/Documents/RenAIssance/Test1/
 ↪llm_corrected_results_final.csv"
eval_df = pd.read_csv(results_path)

# Keep only required columns
eval_df = eval_df[["ground_truth_text", "llm_correction"]].copy()
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].fillna("").
 ↪astype(str)
eval_df["llm_correction"] = eval_df["llm_correction"].fillna("").astype(str)

# Remove BOM if present
eval_df["ground_truth_text"] = eval_df["ground_truth_text"].str.
 ↪replace("\ufeff", "", regex=False)
eval_df["llm_correction"] = eval_df["llm_correction"].str.replace("\ufeff", "",␣
 ↪regex=False)

refs = eval_df["ground_truth_text"].tolist()
hyps = eval_df["llm_correction"].tolist()
```

```python
# Per-line metrics after LLM correction
eval_df["cer_after_recalc"] = [jiwer.cer(r, h) for r, h in zip(refs, hyps)]
eval_df["wer_after_recalc"] = [jiwer.wer(r, h) for r, h in zip(refs, hyps)]

# Corpus-level metrics after LLM correction
corpus_cer_after = jiwer.cer("\n".join(refs), "\n".join(hyps))
corpus_wer_after = jiwer.wer("\n".join(refs), "\n".join(hyps))
chrf_metric = CHRF()
corpus_chrf_after = chrf_metric.corpus_score(hyps, [refs]).score

print(f"Rows evaluated: {len(eval_df)}")
print(f"CER after LLM (corpus):  {corpus_cer_after:.6f}")
print(f"WER after LLM (corpus):  {corpus_wer_after:.6f}")
print(f"chrF after LLM (corpus): {corpus_chrf_after:.4f}")

display(eval_df[["ground_truth_text", "llm_correction", "cer_after_recalc",␣
 ↪"wer_after_recalc"]].tail())
```

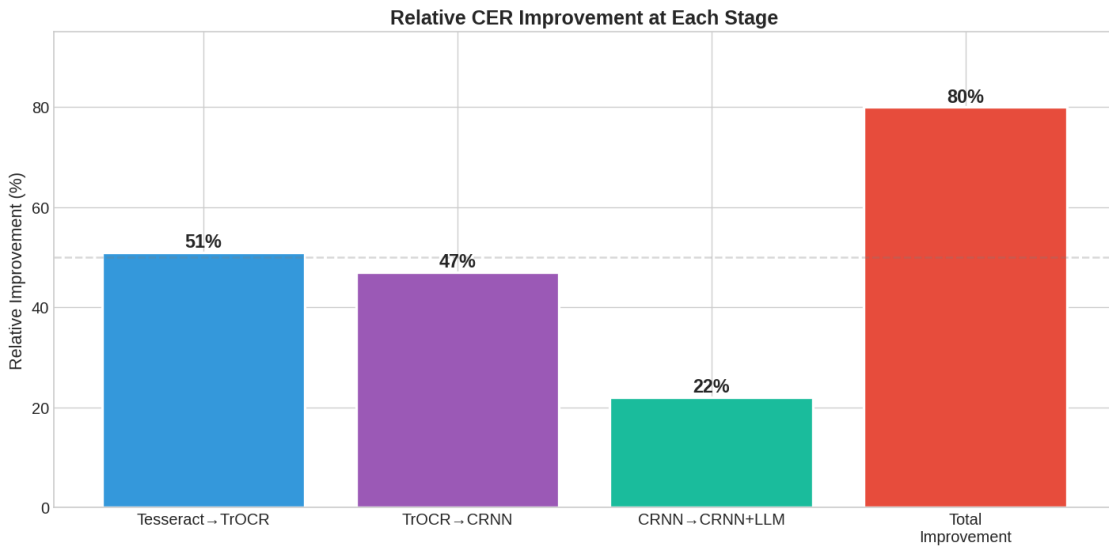
```
Rows evaluated: 296
CER after LLM (corpus):  0.072391
WER after LLM (corpus):  0.236297
chrF after LLM (corpus): 84.9286

                                    ground_truth_text  \
291  sar la injuria hecha a su casa, ex Alex. ad Ba…
292  item ait, ff. de iniurijs: vbi vero, la ofensa…
293  marido y muger, lo es tambien el derecho de ac…
294  puede la muger suplir la omission del marido, …
295  bro in §. patitur, nu. 4. inst. de iniurijs, F…

                                      llm_correction  cer_after_recalc  \
291  sar la injuria hecha a su casa, ex dies ad Bar…          0.131148
292  titem ait, sic de iniurijs,: vbi vero, la oste…          0.122807
293  marido y muger, lo es tambien el derecho de ac…          0.000000
294  puede la muger suplir la omission del marido, …          0.038462
295  bro in § patitur, num. 4. inst. de iniuria. Fa…          0.114754

     wer_after_recalc
291          0.266667
292          0.333333
293          0.000000
294          0.200000
295          0.416667
```

[70]:
```python
# METRICS COMPARISON - BAR CHARTS
```

```
stages = ['Tesseract\nBaseline', 'TrOCR\nFine-tuned', 'CRNN\n(Kraken)', 'CRNN␣
↪+\nLLM Correction']
cer_vals = [0.3553, 0.1750, 0.0928, 0.0724]
wer_vals = [0.7713, 0.4657, 0.3335, 0.2363]
chrf_vals = [0.4850, 0.6507, 0.7837, 0.8493]

fig, axes = plt.subplots(1, 3, figsize=(20, 6))
colors = ['#e74c3c', '#e67e22', '#2ecc71', '#27ae60']

# CER
bars = axes[0].bar(stages, cer_vals, color=colors, edgecolor='white',␣
↪linewidth=2)
axes[0].set_title('Character Error Rate (CER) ↓', fontweight='bold',␣
↪fontsize=14)
axes[0].set_ylabel('CER')
for bar, val in zip(bars, cer_vals):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                 f'{val:.4f}', ha='center', fontweight='bold', fontsize=11)
axes[0].set_ylim(0, max(cer_vals) * 1.15)

# WER
bars = axes[1].bar(stages, wer_vals, color=colors, edgecolor='white',␣
↪linewidth=2)
axes[1].set_title('Word Error Rate (WER) ↓', fontweight='bold', fontsize=14)
axes[1].set_ylabel('WER')
for bar, val in zip(bars, wer_vals):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{val:.4f}', ha='center', fontweight='bold', fontsize=11)
axes[1].set_ylim(0, max(wer_vals) * 1.15)

# chrF
bars = axes[2].bar(stages, chrf_vals, color=colors, edgecolor='white',␣
↪linewidth=2)
axes[2].set_title('chrF Score ↑', fontweight='bold', fontsize=14)
axes[2].set_ylabel('chrF')
for bar, val in zip(bars, chrf_vals):
    axes[2].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                 f'{val:.4f}', ha='center', fontweight='bold', fontsize=11)
axes[2].set_ylim(0, 1.0)

plt.suptitle('OCR Pipeline Performance - Stage-by-Stage Comparison',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

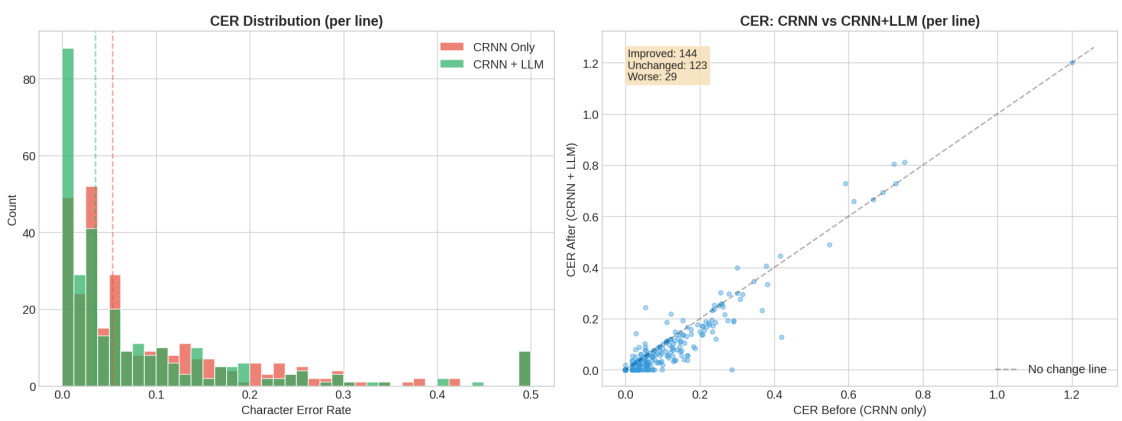


[71]:
```
#
# RELATIVE IMPROVEMENT WATERFALL
#

transitions = ['Tesseract→TrOCR', 'TrOCR→CRNN', 'CRNN→CRNN+LLM',␣
 ↪'Total\nImprovement']
cer_improvements = [51, 47, 22, 80]  # percentage relative improvements

fig, ax = plt.subplots(figsize=(12, 6))
colors_wf = ['#3498db', '#9b59b6', '#1abc9c', '#e74c3c']
bars = ax.bar(transitions, cer_improvements, color=colors_wf,␣
 ↪edgecolor='white', linewidth=2)
for bar, val in zip(bars, cer_improvements):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
            f'{val}%', ha='center', fontweight='bold', fontsize=14)

ax.set_title('Relative CER Improvement at Each Stage', fontweight='bold',␣
 ↪fontsize=15)
ax.set_ylabel('Relative Improvement (%)', fontsize=13)
ax.set_ylim(0, 95)
ax.axhline(y=50, color='gray', linestyle='--', alpha=0.3)
plt.tight_layout()
plt.show()
```



[72]:
```
#
# CER DISTRIBUTION - BEFORE vs AFTER LLM
#

if df_final is not None and 'cer_before' in df_final.columns and 'cer_after' in␣
↪df_final.columns:
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    # Histogram
    axes[0].hist(df_final['cer_before'].clip(0, 0.5), bins=40, alpha=0.7,
                 label='CRNN Only', color='#e74c3c', edgecolor='white')
    axes[0].hist(df_final['cer_after'].clip(0, 0.5), bins=40, alpha=0.7,
                 label='CRNN + LLM', color='#27ae60', edgecolor='white')
    axes[0].set_title('CER Distribution (per line)', fontweight='bold',␣
↪fontsize=14)
    axes[0].set_xlabel('Character Error Rate')
    axes[0].set_ylabel('Count')
    axes[0].legend(fontsize=12)
    axes[0].axvline(x=df_final['cer_before'].median(), color='#e74c3c',␣
↪linestyle='--', alpha=0.5)
    axes[0].axvline(x=df_final['cer_after'].median(), color='#27ae60',␣
↪linestyle='--', alpha=0.5)

    # Scatter: before vs after
    axes[1].scatter(df_final['cer_before'], df_final['cer_after'],
                    alpha=0.4, s=20, color='#3498db')
    lims = [0, max(df_final['cer_before'].max(), df_final['cer_after'].max()) *␣
↪1.05]
```

```
    axes[1].plot(lims, lims, 'k--', alpha=0.3, label='No change line')
    axes[1].set_title('CER: CRNN vs CRNN+LLM (per line)', fontweight='bold',␣
↪fontsize=14)
    axes[1].set_xlabel('CER Before (CRNN only)')
    axes[1].set_ylabel('CER After (CRNN + LLM)')
    axes[1].legend()

    improved = (df_final['cer_after'] < df_final['cer_before']).sum()
    same = (df_final['cer_after'] == df_final['cer_before']).sum()
    worse = (df_final['cer_after'] > df_final['cer_before']).sum()
    axes[1].text(0.05, 0.95, f'Improved: {improved}\nUnchanged: {same}\nWorse:␣
↪{worse}',
                 transform=axes[1].transAxes, fontsize=11,␣
↪verticalalignment='top',
                 bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

    plt.tight_layout()
    plt.show()

    print(f"\n LLM Correction Impact:")
    print(f"   Lines improved:  {improved}/{len(df_final)} ({improved/
↪len(df_final)*100:.1f}%)")
    print(f"   Lines unchanged: {same}/{len(df_final)} ({same/len(df_final)*100:
↪.1f}%)")
    print(f"   Lines worsened:  {worse}/{len(df_final)} ({worse/
↪len(df_final)*100:.1f}%)")
else:
    print("CER before/after columns not available for distribution plot.")
```




```
LLM Correction Impact:
  Lines improved:  144/296 (48.6%)
```

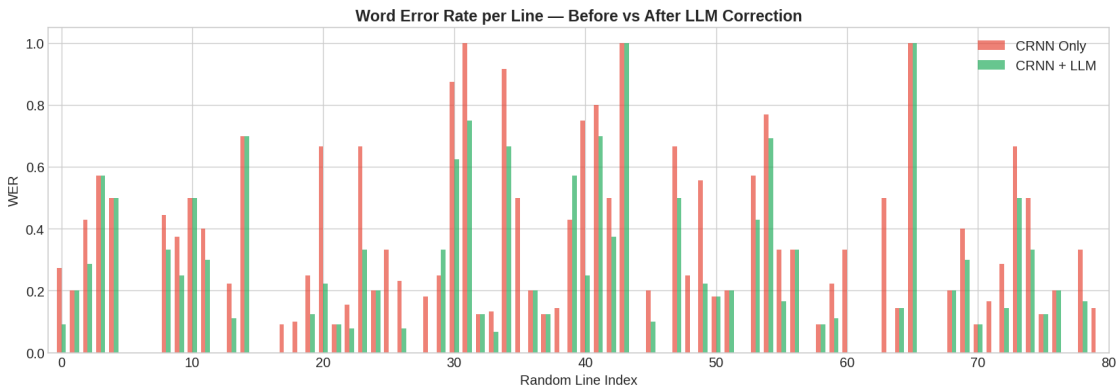
```
Lines unchanged: 123/296 (41.6%)
Lines worsened:  29/296 (9.8%)
```

[73]:
```
#
# SAMPLE PREDICTIONS TABLE
#

if df_final is not None:
    print("Sample Predictions - Ground Truth vs CRNN vs LLM Correction\n")

    # Pick interesting examples (mix of good and challenging)
    sample_idx = list(range(30, min(45, len(df_final))))
    sample_df = df_final.iloc[sample_idx][['ground_truth_text',␣
↪'crnn_prediction', 'llm_correction',
                                           'cer_before', 'cer_after']].copy()
    sample_df.columns = ['Ground Truth', 'CRNN Prediction', 'LLM Corrected',␣
↪'CER Before', 'CER After']
    sample_df['CER Before'] = sample_df['CER Before'].map('{:.3f}'.format)
    sample_df['CER After'] = sample_df['CER After'].map('{:.3f}'.format)

    display(sample_df.style.set_properties(**{'text-align': 'left', 'font-size':
↪ '11px'})
            .set_table_styles([{'selector': 'th', 'props': [('text-align',␣
↪'center')]}]))
```

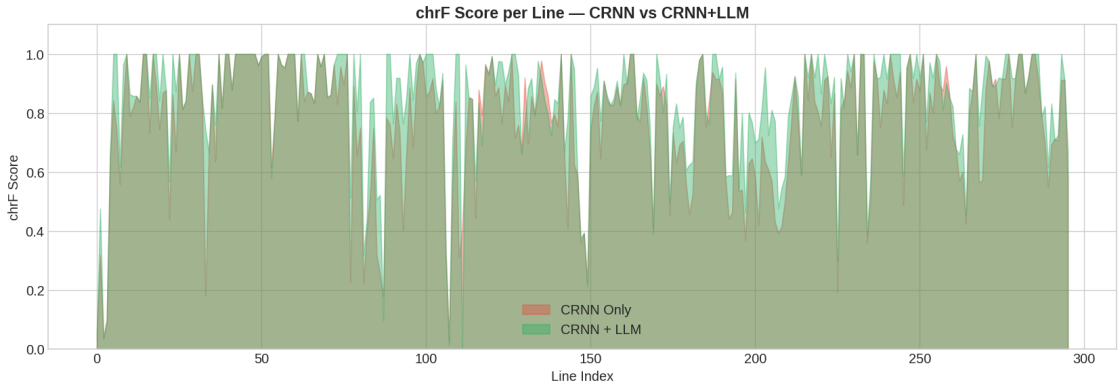
```
Sample Predictions - Ground Truth vs CRNN vs LLM Correction


<pandas.io.formats.style.Styler at 0x7514884ecf90>
```

[74]:
```
#
# WER DISTRIBUTION
#

if df_final is not None and 'wer_before' in df_final.columns:
    # Randomly select 80 rows (or fewer if dataframe has less than 80 rows)
    df_sample = df_final.sample(n=min(80, len(df_final)), random_state=42).
↪reset_index(drop=True)

    fig, ax = plt.subplots(figsize=(14, 5))

    positions = np.arange(len(df_sample))
    width = 0.35

    ax.bar(positions - width/2, df_sample['wer_before'].clip(0, 2), width,
           label='CRNN Only', color='#e74c3c', alpha=0.7)

    ax.bar(positions + width/2, df_sample['wer_after'].clip(0, 2), width,
```

```
            label='CRNN + LLM', color='#27ae60', alpha=0.7)

    ax.set_title('Word Error Rate per Line - Before vs After LLM Correction',
                 fontweight='bold', fontsize=14)

    ax.set_xlabel('Random Line Index')
    ax.set_ylabel('WER')
    ax.legend(fontsize=12)

    ax.set_xlim(-1, len(df_sample))

    plt.tight_layout()
    plt.show()
```



[75]:
```
# chrF IMPROVEMENT
if df_final is not None and 'chrf_before' in df_final.columns:
    fig, ax = plt.subplots(figsize=(14, 5))

    ax.fill_between(range(len(df_final)), df_final['chrf_before'].values,
                    alpha=0.4, color='#e74c3c', label='CRNN Only')
    ax.fill_between(range(len(df_final)), df_final['chrf_after'].values,
                    alpha=0.4, color='#27ae60', label='CRNN + LLM')
    ax.set_title('chrF Score per Line - CRNN vs CRNN+LLM', fontweight='bold',␣
 ↪fontsize=14)
    ax.set_xlabel('Line Index')
    ax.set_ylabel('chrF Score')
    ax.legend(fontsize=12)
    ax.set_ylim(0, 1.1)
    plt.tight_layout()
    plt.show()

    print(f"\n  Mean chrF before: {df_final['chrf_before'].mean():.4f}")
    print(f"  Mean chrF after:  {df_final['chrf_after'].mean():.4f}")
```




```
Mean chrF before: 0.7762
Mean chrF after:  0.8401
```

0.14 13. Conclusions

1. **80% CER reduction** from Tesseract baseline (0.3553 → 0.0724)
2. **Each pipeline stage adds measurable value**: the cascaded approach outperforms any single model
3. **Source-level validation** ensures results generalise to completely unseen manuscripts
4. **LLM post-correction** provides the final polish and fixes character-level OCR errors while keeping historical spelling intact